

Chapter 0: Introduction

Gregory S. DeLozier, PhD

You already know something about writing programs. You can take a problem, choose a representation, and write code that produces a result. This course builds on that experience by asking what happens when other people need to use the software, other developers need to change it, and somebody needs it delivered by a particular date.

Those conditions affect almost every decision. A feature can work exactly as written and still fail to solve the user's problem. A good design can arrive too late to be useful. A program can pass its tests on one developer's computer and fail when the team tries to deploy it.

Software engineering is the work of developing and maintaining useful software under those conditions. It includes programming, but it also includes deciding what to build and finding evidence that it works. Much of the difficulty comes from making those decisions with incomplete information.

We will learn about that work by building a software product. As the project develops, we will have real decisions to examine. Some approaches will work well. Others will need to change, and we will need to understand why.

What do we mean by engineering?

Several definitions approach engineering from different directions. Reading them together helps explain what we will practice in this course.

The definition hosted by IEEE

The Institute of Electrical and Electronics Engineers (IEEE) hosts a definition credited to the Accreditation Board for Engineering and Technology (ABET). The definition reads:

The profession in which a knowledge of the mathematical or physical sciences gained by study, experience and practice is applied with judgement to develop ways to utilize, economically, the materials and forces of nature for the benefit of mankind. (1)

Notice the connection between knowledge and judgment. Knowing a principle is useful, but someone still has to decide how it applies to the problem at hand. The words “economically” and “benefit” also matter. Engineering has to accomplish something worthwhile with the resources available.

The Wikipedia definition

Wikipedia defines engineering this way:

Engineering is the practice of systematically applying natural science and mathematics to design and improve systems, devices, or processes that solve problems under constraints. (2)

This definition makes the connection explicit: technical knowledge is applied to a problem, and the solution must work within limits. The work may improve something that already exists; it does not have to produce an entirely new invention.

Our working definition

For this class, we will use this working definition:

Engineering is the application of known principles and the orderly gathering of new knowledge to predictably create a product within the bounds of resource constraints.

“Predictably” does not mean that nothing will surprise us. It means we should have reasons to expect the approach to work, and ways to investigate when it does not. An experiment can be engineering work when it answers a useful question. Repeatedly changing things without understanding the results gives us much less to rely on.

Science, a real problem, and constraints

Three ideas connect these definitions. They are expressed differently, and some are more explicit in one definition than another, but together they give us a useful way to examine our work.

Part of the engineering problem	Question to ask
Science or technology	What knowledge can we apply, and what do we need to find out?
A real problem	Whose situation should improve, and what would count as improvement?
Constraints	What can we deliver with the time and resources available? What conditions must it satisfy?

Apply what we know, and learn what we need

Software depends on technical knowledge. An algorithm has costs. A network request can fail. Two people may try to change the same data at nearly the same time. These facts affect whether a design will work even when the interface looks convincing.

We do not have to invent every solution ourselves. Established techniques and reliable tools give us a starting point. We still need to check that they fit our situation. When we are unsure whether a service can handle the expected demand, a small measurement can be more useful than an argument based on preference.

Gathering knowledge takes effort, so choose the question deliberately. A test should tell us something that could affect a decision. We need enough evidence to take a sensible next step, without making complete certainty a condition for starting.

Solve a real problem

Technical success and practical success are connected, but they are not identical. A reservation page may save records correctly while leaving students unsure whether they have permission to collect the equipment. The code works; the uncertainty that brought people to us remains.

A real problem need not be large or dramatic. Avoiding a wasted trip is a useful result. So is reducing the time a staff member spends correcting reservations. We need to understand what someone is trying to accomplish well enough to recognize whether our work helps.

This also gives us a reason to question an attractive feature. A calendar animation might demonstrate technical skill, but preventing conflicting reservations may do more for the people using the system.

Work within constraints

A solution has to fit the situation in which it will be used. A deadline limits the work we can finish. A budget limits what we can buy or operate. The people who maintain the result have limited time too.

Other constraints concern acceptable behavior. A system may need to protect private information or preserve confirmed reservations after a restart. These are part of making the solution usable and dependable, even when they add work.

Constraints require choices. We might postpone an optional feature to leave time to check recovery after a failure. Removing a privacy protection to meet a deadline would have different consequences. Engineering judgment includes explaining those consequences and finding a scope that can be delivered responsibly.

What makes it software engineering?

Software engineering applies this way of working to software throughout its useful life. Wikipedia describes a field that includes development and maintenance, with attention to what the software must do and how it is built. (3)

A widely cited definition from the *IEEE Standard Glossary of Software Engineering Terminology*, IEEE 610.12-1990, describes:

The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software. (4)

Development is only part of that definition. Someone must operate the software, and someone must change it as needs change. In this class, our working definition therefore applies to the decisions around the code as well as the code itself. We use technical knowledge and deliberate investigation to deliver useful software within real limits, then keep it useful.

The following equipment reservation example brings those ideas together.

Useful software starts with a problem

Suppose a campus organization lends equipment to students. Reservations currently arrive by email. Someone checks a spreadsheet, replies to the student, and updates the reservation. Occasionally two people are promised the same item.

A team could immediately start building a reservation application. It could choose a framework, create a database, and put a calendar on a page. All of that might eventually be useful. First, though, the team needs to understand what is going wrong.

Perhaps the real difficulty is that several staff members update separate copies of the spreadsheet. Perhaps equipment is returned late, so the reservation calendar is accurate but the item is unavailable. Perhaps students cannot tell whether their requests have been accepted. Returned equipment may also need time for inspection or charging before it can be lent again. This gap between uses is called turnaround time. The team needs to find out whether reservations must leave room for that work.

Each explanation suggests different work. A shared schedule might address one problem. Another might require a check-in process. A confirmation message could matter more to students than a more elaborate calendar.

Before committing to a solution, we need to understand whose problem we are solving. We also need some way to tell whether we have improved the situation. That is why the course begins with users and requirements.

The same reasoning applies to business needs. A useful product has to justify the effort spent creating and operating it. That justification might be revenue, but it might also be less staff time spent resolving mistakes. A campus service can have a strong reason to exist without charging its users.

Working within limits

Now suppose the organization needs a usable system before equipment checkout begins in four weeks. The team has other classes and limited experience with deployment. Those facts are part of the engineering problem.

A proposal for every possible feature is not yet a delivery plan. The team needs to decide what it can finish and what can wait. It also needs to identify the uncertainties that could prevent delivery.

For the reservation system, preventing two people from claiming the same item might be essential. Color choices for the calendar can probably wait. Whether the application can use the organization's sign-in system is an uncertainty worth investigating early, because the answer could change the design and the amount of work.

We will return to this kind of judgment throughout the course. A practice is useful when it helps us make a decision, coordinate work, or catch a mistake at a reasonable cost. A short experiment may answer a question better than a long planning document. A written decision may save several people from repeating the same argument next week.

There is no promise that following a particular process will make every project succeed. We will examine what a practice is supposed to accomplish and whether it is helping in the situation at hand.

The course project

The project connects the course topics. Requirements become work that a team can plan. Design decisions affect how easily that work can be divided. Tests provide evidence about behavior. Deployment makes the result available to somebody who is not running the development environment.

The project also makes gaps visible. An application may have a working interface while its data disappears on restart. Two features may work separately and fail when combined. The person who wrote a component may be the only person who knows how to run it.

These are useful discoveries. They give us specific problems to solve and reasons to improve the way the team works.

We will return to the product in several presentations and project checkpoints. Each is a chance to see what works while there is still time to change it. Show what a person can actually do with the current version. Be clear about what is unfinished and what needs attention next.

For the reservation example, a meaningful demonstration might follow one request through confirmation and then attempt a conflicting reservation. That tells us more about the product than a tour of unfinished screens. It also gives the team a clear boundary for a small piece of work.

The project assignments will provide the specific requirements and submission instructions.

How the course develops

We start by deciding what is worth building. That means understanding users and their requirements, then considering what is possible within the available time. User stories give us a way to describe intended uses. Acceptance criteria make the expected behavior specific enough to check.

For example, “students can reserve equipment” leaves many questions open. “A confirmed reservation prevents another student from reserving the same item for an overlapping period” gives the team a behavior it can discuss and test. Planning becomes easier when the work is that clear.

Next comes the work of building together. We will study agile development and how teams adjust their plans as they learn. We will use version control and GitHub workflow to manage changes. Design and modularity help us divide the software into parts that people can understand and work on. Test-driven development gives us a way to develop behavior by writing a test, making it pass, and improving the code. At the midterm checkpoint, we will look at how these ideas are working in the project.

Delivery gets its own attention. Getting an application to run on a development computer is only part of making it available to users. Deployment requires a suitable environment and a repeatable way to put the software there. We will look for work we can automate, especially work that is easy to get wrong by hand. Continuous integration brings changes together frequently and checks the combined result. Continuous delivery aims to keep the software ready for release. The decision to release can still be a separate step.

As the product approaches launch, we will ask harder questions about readiness. Can another person use it without the developer sitting beside them? Can the team identify which version is running? Can we recover when a deployment fails? Implementation and review continue while we work through those questions.

The later topics concern software that has to keep working and keep changing. Configuration and version management help us control what is running. Maintenance and evolution bring us back to earlier design decisions: which ones made change easier, and which ones created extra work? Architecture and quality give us ways to investigate those consequences. We will also examine the team’s process and how consistently it supports good work.

Economics, professional practice, and ethics round out the course. A feature consumes time that could have gone toward something else. A shortcut may leave work for the next person. A product may collect information its users would rather keep private. These concerns belong in our decisions throughout the project, even though we study them more directly near the end.

Working as a team

A team needs enough shared understanding that progress does not depend on one person’s memory. Other people need to be able to find the code, understand the current work, and reproduce the steps needed to run it.

Version control helps preserve changes. An issue tracker gives a problem or proposed feature a place to be discussed. A brief setup document can let a teammate start working without waiting for its author to become available. These practices address ordinary coordination problems.

A small team may share the work that a larger organization assigns to separate people. The responsibilities still exist. Someone has to establish what the product should do, and the team needs evidence that it does those things. Deployment and operation need attention too. Dividing up the coding does not automatically cover all of that work.

Sharing responsibilities also means making uncertainty visible. If a feature depends on an unfamiliar service, the team needs to know that before the deadline. If a test is failing, hiding it does not make the product more complete. A precise description of an unfinished task is often more useful than an optimistic estimate of how close it is to being done.

Review helps distribute understanding. Explaining a change to another person can reveal an assumption that was never written down. Reading someone else's change can expose a dependency before it becomes a failure in the combined application.

Work you can enjoy doing

This course is also about having an enjoyable career. Building something useful can be satisfying work. So can solving a difficult problem with people you trust. The way a team organizes its work has a great deal to do with whether those experiences are common or rare.

Consider two teams using the same programming language and development tools. On one team, priorities change without anyone adjusting the deadline. Important decisions happen in private conversations. A broken deployment becomes an emergency for the one person who knows how it works. People spend their time recovering from surprises and trying to avoid being blamed for the next one.

On the other team, a change in priority comes with a decision about what to postpone. Decisions are recorded where the people doing the work can find them. Several people understand deployment, and a failed release has a known recovery procedure. Problems still happen. The team has ways to deal with them without making every problem a personal crisis.

The tools alone cannot account for that difference. A better editor will not resolve contradictory priorities. A new framework will not make an impossible deadline realistic. Those problems require decisions about how people work together.

Good engineering practices should help people do work they can take pride in at a pace they can sustain. Review can make a difficult change less lonely. Repeatable tests can reduce the anxiety of touching unfamiliar code. Clear scope can let someone finish a task without discovering that everyone expected something different.

Process can also become part of the problem. A meeting that never produces a decision consumes time people need for the work. An approval step with no clear purpose can leave a finished change waiting for days. We need to evaluate those practices as carefully as we evaluate the code. What difficulty does the practice address? Is it helping enough to justify the effort?

As the project develops, pay attention to how the work is going for the people doing it. Repeated interruptions, confusing handoffs, and avoidable emergencies are evidence worth examining. Learning to recognize and improve those conditions is part of becoming a software engineer. It is also useful preparation for choosing a team and building a career you want to stay in.

Reading and using the course materials

The workbook provides the sustained explanation behind the course. Its chapters follow the syllabus, so the reading connects to the work being discussed and practiced. Examples introduce ideas in a limited setting; the project will usually require judgment about how those ideas apply.

The slides serve a different purpose. They provide examples and structure for the class discussion. A short slide may depend on an explanation that belongs in the workbook. Reading only the slides will not always provide that explanation.

Alongside the workbook, we will use *Head First Software Development* by Dan Pilone and Russ Miles, published by O'Reilly (5). It is available at no cost through the university library. The book works through software development using concrete examples, giving us another way to examine the problems that come up in the project.

We will also use the *Guide to the Software Engineering Body of Knowledge*, abbreviated SWEBOK, as a professional reference (6). It gives us a broader view of the field alongside the project work. The *Scrum Guide* by Ken Schwaber and Jeff Sutherland provides a reference for the Scrum framework discussed in the course (7). Supplemental readings provide further explanations and examples.

Read with a concrete question in mind. When studying a development practice, ask what problem it addresses and what evidence would show that it is helping. When studying a tool, connect its operations to the team's work. Being able to perform a command is useful; knowing when it is appropriate is part of learning to use it well.

Responsibility for the result

Useful software creates responsibilities. A reservation system might expose information about who has borrowed equipment. An incorrect confirmation could cause someone to arrive expecting an item that is not available. Even a small application can affect somebody else's time and decisions.

Testing cannot establish that every possible failure is absent. A successful demonstration cannot establish that an application is secure. Engineering judgment includes being accurate about what has been checked and what remains unknown.

That accuracy matters within the team as well. A teammate deciding whether to release a feature needs an honest account of its condition. Clear limitations make it possible to choose the next test or reduce the scope of a release.

By the end of the course, you should be better able to connect a software decision to a user's need, explain the tradeoffs, and support your conclusions with evidence. You should also have a clearer idea of the working conditions that help you do good work and enjoy doing it. We will begin by looking more closely at software engineering as practical problem solving: understanding a situation well enough to choose useful work.

References

1. IEEE. "What is Engineering?" Presents the ABET definition. <https://ewh.ieee.org/cmte/pa/UCF/Engineering.html>.
2. Wikipedia contributors. "Engineering." <https://en.wikipedia.org/wiki/Engineering>.
3. Wikipedia contributors. "Software engineering." https://en.wikipedia.org/wiki/Software_engineering.
4. IEEE. *IEEE Standard Glossary of Software Engineering Terminology*. IEEE Std 610.12-1990, 1990. <https://doi.org/10.1109/IEEESTD.1990.101064>.
5. Dan Pilone and Russ Miles. *Head First Software Development*. O'Reilly Media, 2007. <https://www.oreilly.com/library/view/head-first-software/9780596527358/>.

6. IEEE Computer Society. *Guide to the Software Engineering Body of Knowledge (SWEBOK Guide)*, Version 4. <https://www.computer.org/education/bodies-of-knowledge/software-engineering/v4>.
7. Ken Schwaber and Jeff Sutherland. *The Scrum Guide*, November 2020. <https://scrumguides.org/scrum-guide.html>.

Questions for Thought

1. In the working definition, why do we need both known principles and new knowledge? Describe a software decision for which each would matter.
2. A reservation application passes every test the team wrote, but students still cannot tell whether their requests are confirmed. What has the team established, and what does it still need to investigate?
3. Choose a small software product you know. Identify the real problem it addresses and one constraint that could change how you would build it.
4. How might a team's way of working make the same technical task enjoyable or frustrating? Describe one change that would help without replacing its tools.